

Copilot Chat Observability

Guia completo de implantação

Cliente:	Equifax Boa Vista
Solução:	Dynatrace AI & LLM Observability
Escopo:	GitHub Copilot Chat nas IDEs (VS Code + CLI)
Tenant:	https://fov31014.apps.dynatrace.com
Data:	17 de julho de 2026
Versão:	1.0

Sumário

1. Contexto e objetivo
2. Resultados comprovados na POC
3. Arquitetura da solução
4. Pré-requisitos
5. Passo 1 — Configurar o Dynatrace
6. Passo 2 — Subir o OTEL Collector
7. Passo 3 — Configurar o GitHub Enterprise
8. Passo 4 — Piloto no VS Code
9. Passo 5 — Validar no Dynatrace
10. Passo 6 — Consumir os dados
11. Considerações de LGPD
12. Rollback e troubleshooting
13. Cronograma e responsabilidades
14. Referências oficiais

1 Contexto e objetivo

A Boa Vista solicitou visibilidade em tempo real do consumo do GitHub Copilot pelos desenvolvedores, cobrindo **quatro pilares**:

1. **FinOps** — tokens consumidos e custo estimado, com breakdown por usuário, time e sessão
2. **Performance (SRE)** — latência das requisições aos modelos e identificação de gargalos
3. **Erros e limites** — visibilidade sobre *rate limits* (HTTP 429) e falhas de comunicação
4. **Adoção** — quais modelos estão sendo mais usados e por quais desenvolvedores

Esta solução implanta um pipeline OpenTelemetry nativo que captura, em tempo real, todas as interações dos devs com o Copilot Chat no VS Code, envia para o Dynatrace, e disponibiliza os dados na app nativa **AI & LLM Observability** — sem que nenhum dev precise fazer configuração manual.

Por que agora é possível

A Microsoft/GitHub lançaram em **julho de 2026**

o suporte OpenTelemetry nativo para o Copilot Chat no VS Code (*Monitor agent usage with OpenTelemetry*) e o *Enterprise managed-settings.json* em GA, permitindo enforcement centralizado. Isso viabiliza pela primeira vez a captura em tempo real do consumo do Copilot na IDE, com granularidade de *span* por chamada.

2 Resultados comprovados na POC

Durante a construção desta POC, com **apenas uma máquina** (a do consultor Dynatrace) emitindo telemetria, capturamos os seguintes dados em 24 horas:

Custo real capturado

US\$ 227,65

Chats + Tool calls

41 + 28

Em 26 sessões distintas de Copilot Chat

Agents descobertos

7

Sub-agents que consomem tokens silenciosamente

21 chamadas ao **claude-opus-4.7** em 24h — uma única máquina

🌟 Insight de FinOps

Uma única pessoa consumiu

US\$ 227,65

em 24h com Copilot. Extrapolando para **47 devs**

da Boa Vista com padrão similar de uso, o custo diário estimado é de **~US\$ 10.700**

, ou aproximadamente

US\$ 320 mil/mês

. Sem observabilidade, esse número é invisível — vocês veem apenas a fatura agregada.

Custo total por modelo — dados reais capturados

💰 FinOps — Consumo e custo

Custo é estimado via pricing hardcoded. Ajustar conforme contrato Copilot Business/Enterprise.

Custo total por modelo (últimas 24h)

Run

Set in query

Hide input

Maximize

Options

```
1 fetch spans, from:now()-24h
2 | filter service.name == "copilot-chat"
3 | filter isNotNull(gen_ai.usage.input_tokens)
4 | fieldsAdd inp = toDouble(gen_ai.usage.input_tokens)
5 | fieldsAdd out = toDouble(gen_ai.usage.output_tokens)
6 | fieldsAdd cost_usd =
7 |   if(contains(gen_ai.request.model, "opus"),
8 |     (inp * 0.000015) + (out * 0.000075),
9 |     else: if(contains(gen_ai.request.model, "mini"),
10 |      (inp * 0.0000015) + (out * 0.000006),
11 |      else: if(contains(gen_ai.request.model, "claude"),
12 |       (inp * 0.000003) + (out * 0.000015),
13 |       else: if(contains(gen_ai.request.model, "gpt-4o"),
14 |        (inp * 0.0000025) + (out * 0.00001),
15 |        else: (inp * 0.000002) + (out * 0.00001))))))
16 | summarize total_cost = sum(cost_usd),
17 |           input_tokens = sum(inp),
18 |           output_tokens = sum(out),
19 |           requests = count(),
20 |           by:{gen_ai.request.model}
21 | sort total_cost desc
```

3 records Executed at: 17/07/2026, 17:54:32, Timeframe: 16/07/2026, 17:54:32 - 17/07/2026, 17:54:32, Scanned bytes: 358 MB

gen_ai.request.model	total_cost	input_tokens	output_tokens	requests
claude-opus-4.7	227.65	14,840,662.00	67,180.00	21
gpt-4o-mini-2024-07-18	< 0.01	35,783.00	705.00	21
gpt-4o	< 0.01	342.00	189.00	1

Cell 3 do notebook — **claude-opus-4.7** respondeu por US\$ 227,65 em 21 chamadas; sub-agents em **gpt-4o-mini** serviram 21 tarefas auxiliares por menos de US\$ 0,01.

Volume nas últimas 24h

 Volume — chats vs execuções de tool (últimas 24h)

```
1 fetch spans, from:now()-24h
2 | filter service.name == "copilot-chat"
3 | summarize chats = countIf(startsWith(span.name, "chat ")),
4               tools = countIf(startsWith(span.name, "execute_tool ")),
5               sessions = countDistinct(gen_ai.conversation.id)
```

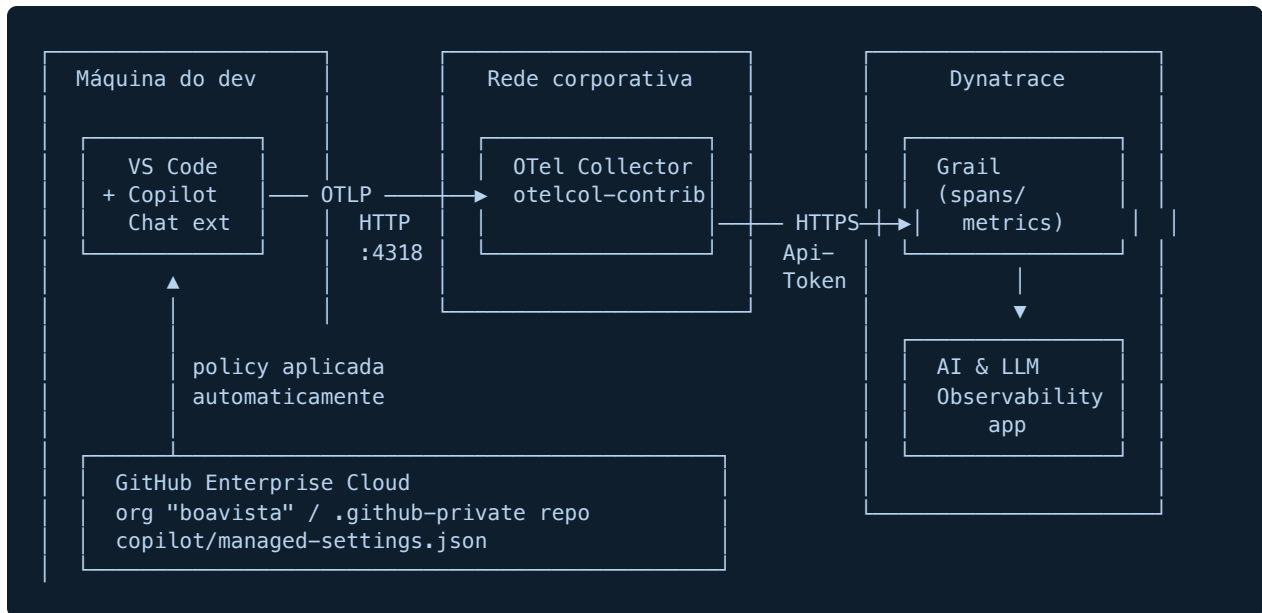
1 record

Executed at: 17/07/2026, 17:54:26, Timeframe: 16/07/2026, 17:54:26 - 17/07/2026, 17:54:26, Scanned bytes: 357 MB ⓘ

chats	tools	sessions
41	28	26

41 chats principais + 28 execuções de tools distribuídas em 26 sessões distintas.

3 Arquitetura da solução



Componentes

Componente	Papel	Onde roda	Responsável
VS Code + Copilot Chat	Emite spans/metrics OTel de cada chat, tool call, sub-agent	Máquina de cada dev	GitHub Copilot (nativo, GA jul/2026)
managed-settings.json	Ativa e configura OTel em todos os VS Codes automaticamente	Repo <code>.github-private</code> no GHEC	Admin GitHub da Boa Vista
OTel Collector	Recebe telemetria dos devs, adiciona autenticação, forwarda	1 VM/container na rede da Boa Vista	Plataforma / SRE da Boa Vista
Dynatrace AI Observability	Armazena no Grail, detecta automaticamente serviços LLM	SaaS — tenant <code>f ov31014</code>	Admin Dynatrace da Boa Vista

Fluxo de dados

- Dev abre o VS Code — extensão Copilot Chat detecta `github.copilot.chat.otel.enabled=true` via managed policy
- Dev faz uma pergunta ao Copilot Chat — extensão gera árvore de spans (`invoke_agent` → N × `chat` → N × `execute_tool`)
- Spans são enviados via OTLP HTTP para o OTel Collector interno na rede corporativa
- Collector adiciona o header `Authorization: Api-Token dt0c01.xxx` e forwarda via HTTPS para o Dynatrace
- Grail armazena; app AI & LLM Observability detecta automaticamente o serviço `copilot-chat`
- Dashboards, notebook e DQL passam a consultar os dados

4 Pré-requisitos

4.1 Contratos e planos

Item	Requisito
GitHub Copilot	Copilot Business (\$19/dev/mês) ou Copilot Enterprise (\$39/dev/mês). Plano Individual não suporta managed settings.
GitHub Enterprise Cloud (GHEC)	Necessário para publicar o <code>managed-settings.json</code> via server-managed. Alternativa: file-based via MDM ou Ansible.
Dynatrace	Tenant com AI & LLM Observability habilitado (padrão em SaaS moderno). Tenant atual: <code>fov31014</code> .

4.2 Infraestrutura

Item	Requisito
Host do collector	VM Linux (Ubuntu 22.04, RHEL 9, Amazon Linux 2023) ou pod Kubernetes Recursos: 2 vCPU / 2 GB RAM / 10 GB disco
Runtime	Docker, Podman ou containerd
Firewall — ingresso	Porta <code>4318/tcp</code> acessível pelas máquinas dos devs (rede interna, VPN, Zscaler, etc.)
Firewall — egresso	Do collector para <code>*.live.dynatrace.com:443</code> (HTTPS)
DNS interno	Registro apontando para o host, ex.: <code>otel-collector.boavista.corp</code>

4.3 Software nos devs

Item	Requisito
VS Code	Versão 1.128 ou superior (precedência de managed policies)
Extensão	GitHub Copilot Chat atualizada (autoatualização normalmente ativa)
Conta GitHub	Logada em conta membro de uma org GHEC da Boa Vista com licença Copilot Business/Enterprise

5

Passo 1 — Configurar o Dynatrace

Responsável: Admin do tenant Dynatrace · **Tempo estimado:** 15 minutos

5.1 Criar API Token classic para OTLP ingest

1. Acesse o tenant: <https://fov31014.apps.dynatrace.com>
2. `Ctrl+K` → digite **Access Tokens** → **Generate new token**
3. Nome sugerido: `copilot-otlp-ingest`
4. Selecionar os scopes:
 - `openTelemetryTrace.ingest` — para receber spans
 - `metrics.ingest` — para receber metrics
5. **Generate** — o token começa com `dt0c01.` (~96 caracteres)
6. Salvar no cofre corporativo (1Password, HashiCorp Vault, AWS Secrets Manager, etc.)

Segurança

Este token nunca é distribuído para máquinas de devs — fica apenas no OTel Collector. Isso permite rotação sem redeploy nos devs e reduz drasticamente a superfície de exposição.

5.2 (Opcional) Preparar dashboard e notebook

Os assets prontos desta POC estão em:

- `dashboards/copilot-chat-observability.json` — importar via `Ctrl+K` → Dashboards → + **Dashboard** → **Upload**
- Notebook v2 já criado no tenant: [abrir aqui](#)

6

Passo 2 — Subir o OTel Collector

Responsável: Plataforma / SRE da Boa Vista · **Tempo estimado:** 1 dia útil (inclui provisionamento e liberação de firewall)

6.1 Arquivo `compose.yml`

```
services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:latest
    container_name: bv-otel-collector
    restart: unless-stopped
    command: ["--config=/etc/otel-collector-config.yaml"]
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml:ro
    env_file: .env
    ports:
      - "4318:4318"      # OTLP HTTP (VS Code envia aqui)
      - "4317:4317"      # OTLP gRPC (alternativa)
      - "13133:13133"    # Health check
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:13133"]
      interval: 30s
```


6.2 Arquivo `otel-collector-config.yaml`

```
receivers:
  otlp:
    protocols:
      http:
        endpoint: 0.0.0.0:4318
        max_request_body_size: 4194304
      grpc:
        endpoint: 0.0.0.0:4317

processors:
  batch:
    timeout: 5s
    send_batch_size: 512
  resource:
    attributes:
      - key: deployment.environment
        value: production
        action: upsert

exporters:
  otlphttp/dynatrace:
    endpoint: ${env:DT_OTLP_ENDPOINT}
    headers:
      Authorization: "Api-Token ${env:DT_INGEST_TOKEN}"
    compression: gzip

extensions:
  health_check:
    endpoint: 0.0.0.0:13133

service:
  extensions: [health_check]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch, resource]
      exporters: [otlphttp/dynatrace]
    metrics:
      receivers: [otlp]
      processors: [batch, resource]
      exporters: [otlphttp/dynatrace]
```

6.3 Arquivo `.env`

```
# Endpoint OTLP do Dynatrace – nota: live.dynatrace.com (não apps)
DT_OTLP_ENDPOINT=https://fov31014.live.dynatrace.com/api/v2/otlp

# Token gerado no Passo 1
DT_INGEST_TOKEN=dt0c01.XXXXXXXXXXXXXXXXXXXXXXXXXX...
```

6.4 Subir o collector

```
docker compose up -d
docker logs -f bv-otel-collector
```

6.5 Validar

Health check

```
curl http://otel-collector.boavista.corp:13133/
# Deve retornar HTTP 200
```

Teste de span sintético (end-to-end)

```
curl -X POST http://otel-collector.boavista.corp:4318/v1/traces \
-H "Content-Type: application/json" \
-d '{"resourceSpans":[{"scopeSpans":[{"spans":[{"traceId":"00112233445566778899aabbccddeeff",
"spanId":"0011223344556677","name":"test-span",
"startTimeUnixNano":"1721242400000000000",
"endTimeUnixNano":"1721242401000000000"}]}]}]}'
# HTTP 200 = OK. Aguardar 15s e validar no Dynatrace:
# fetch spans | filter span.name == "test-span" | limit 5
```

Responsável: Admin do GitHub Copilot Enterprise da Boa Vista · **Tempo estimado:** 1 dia útil

7.1 Selecionar a organização hospedeira

1. Login no GitHub Enterprise Settings
2. Ir em **AI Controls** → **Agents**
3. Selecionar a organização que hospederá o `.github-private` (geralmente a org corporativa central)

7.2 Criar o repositório `.github-private`

Se ainda não existir, criar um repositório **privado** com esse nome exato na organização selecionada. É onde ficam configurações que se aplicam a toda a org sem serem visíveis publicamente.

7.3 Publicar o arquivo `copilot/managed-settings.json`

Path exato no repositório: `copilot/managed-settings.json`, no branch default (`main`).

```
{
  "telemetry": {
    "enabled": true,
    "endpoint": "https://otel-collector.boavista.corp:4318",
    "protocol": "otlp-http",

    "captureContent": false,
    "lockCaptureContent": true,

    "serviceName": "copilot-chat-boavista",
    "resourceAttributes": {
      "deployment.environment": "production",
      "organization": "boavista",
      "cost_center": "engineering"
    }
  }
}
```

7.4 Referência de campos

Campo	Valor sugerido	Descrição
<code>enabled</code>	<code>true</code>	Liga a exportação OTel do Copilot Chat em todos os VS Codes
<code>endpoint</code>	URL do collector interno	Onde os spans são enviados. HTTPS recomendado.
<code>protocol</code>	<code>otlp-http</code>	HTTP é o padrão. <code>otlp-grpc</code> também suportado.

Campo	Valor sugerido	Descrição
<code>captureContent</code>	<code>false</code>	Não captura texto de prompt/resposta. LGPD-safe. Ver Seção 11.
<code>lockCaptureContent</code>	<code>true</code>	Impede que devs sobrescrevam <code>captureContent</code> individualmente
<code>serviceName</code>	<code>copilot-chat-boavista</code>	Nome do serviço no Dynatrace
<code>resourceAttributes</code>	ver exemplo	Atributos aplicados a todos os spans (ambiente, org, cost center)

7.5 Propagação

Tempo de propagação: até 1h

O VS Code faz pull das managed settings a cada 1h e ao fazer login. Depois de commitar o arquivo, os devs recebem a configuração sem fazer nada. Para forçar antes disso, o dev pode simplesmente reiniciar o VS Code.

7.6 Canais alternativos de delivery

Se a Boa Vista não usa GHEC ou prefere não expor o repositório `.github-private`, há dois canais alternativos:

Canal	Como	Quando usar
File-based	Colocar <code>managed-settings.json</code> em Linux: <code>/etc/github-copilot/</code> macOS: <code>/Library/Application Support/GitHubCopilot/</code> Windows: <code>%ProgramFiles%\GitHubCopilot\</code>	Deploy via Ansible/Chef/Puppet/SCCM
Native MDM	Windows Registry: <code>HKLM\SOFTWARE\Policies\GitHubCopilot</code> macOS: preference domain <code>com.github.copilot</code>	Frotas gerenciadas por Intune, Jamf, GPO

8 Passo 4 — Piloto no VS Code (opcional, para 3-5 devs voluntários antes do rollout enterprise)

Responsável: Dev voluntário · **Tempo estimado:** 5 minutos por dev

Por que fazer piloto?

Enquanto o time de GitHub Admin prepara e publica o `managed-settings.json`, você pode validar a arquitetura com 3-5 devs configurando manualmente no user settings deles. Quando o

rollout enterprise acontecer, o managed-settings sobrescreve automaticamente sem que os devs precisem desfazer nada.

8.1 Configurar o user settings do VS Code

1. `Cmd/Ctrl+Shift+P` → digite **Preferences: Open User Settings (JSON)**
2. Adicionar as 5 linhas de OTel dentro das chaves `{}`:

```
{
  // ...suas outras settings...

  "github.copilot.chat.otel.enabled": true,
  "github.copilot.chat.otel.exporterType": "otlp-http",
  "github.copilot.chat.otel.otlpEndpoint": "http://otel-collector.boavista.corp:4318",
  "github.copilot.chat.otel.captureContent": false,
  "github.copilot.chat.otel.maxAttributeSizeChars": 0
}
```

8.2 Reload do VS Code

`Cmd/Ctrl+Shift+P` → **Developer: Reload Window**

8.3 (Opcional) Enriquecer com identidade do dev

Para popular `user.email`, `user.team`, `cost_center` em cada span, adicionar variável de ambiente no shell:

Fish shell

```
set -Ux OTEL_RESOURCE_ATTRIBUTES \
  "user.email=nome.sobrenome@boavista.com.br,user.name=Nome Sobrenome,user.team=plata
```

Bash / Zsh

```
echo 'export OTEL_RESOURCE_ATTRIBUTES="user.email=nome.sobrenome@boavista.com.br,user
```

Herança da variável no VS Code

No macOS, se o dev abrir o VS Code pelo ícone do dock (Finder), a variável de ambiente do shell NÃO é herdada. Precisa fechar completamente (`Cmd+Q`) e reabrir pelo terminal com `code /caminho/do/workspace`. Windows/Linux herdam normalmente.

8.4 No rollout enterprise (via managed-settings.json)

Nada. Os devs não fazem absolutamente nada. Ao próximo login/reinício do VS Code, a policy é aplicada automaticamente e sobrescreve qualquer user setting anterior. Se o time de Plataforma distribuir também a variável `OTEL_RESOURCE_ATTRIBUTES` via Intune/Jamf/GPO, nem mesmo isso os devs precisam configurar.

9 Passo 5 — Validar no Dynatrace

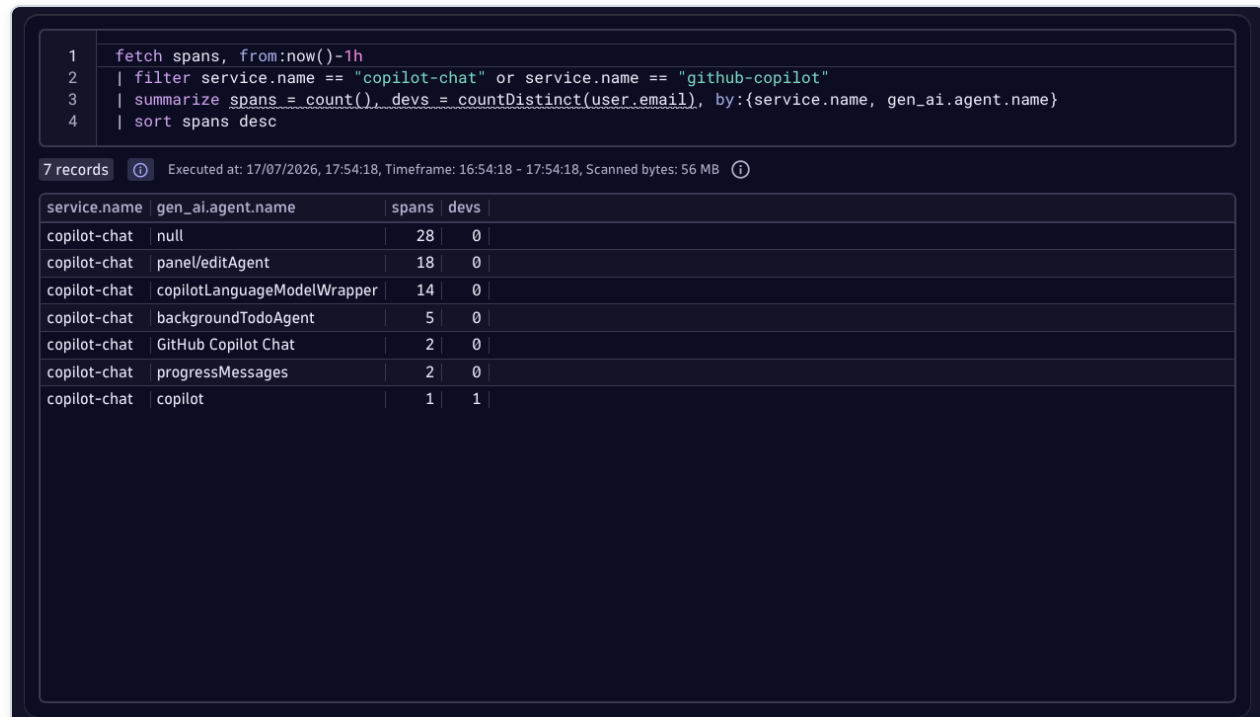
Depois de configurar collector + policy + fazer 2-3 interações reais com Copilot Chat, aguardar ~15 segundos e validar:

9.1 Sanity check via DQL

Abrir o notebook do tenant e rodar:

```
fetch spans, from:now()-1h
| filter service.name == "copilot-chat"
| summarize spans = count(),
              devs = countDistinct(user.email),
              by:{service.name, gen_ai.agent.name}
| sort spans desc
```

Retorno esperado — múltiplos agents do Copilot Chat com spans capturados:



The screenshot shows a Dynatrace notebook interface. At the top, a DQL query is entered in a code editor:

```
1 fetch spans, from:now()-1h
2 | filter service.name == "copilot-chat" or service.name == "github-copilot"
3 | summarize spans = count(), devs = countDistinct(user.email), by:{service.name, gen_ai.agent.name}
4 | sort spans desc
```

Below the query, the execution status is shown: "7 records" and "Executed at: 17/07/2026, 17:54:18, Timeframe: 16:54:18 - 17:54:18, Scanned bytes: 56 MB".

The results are displayed in a table with the following data:

service.name	gen_ai.agent.name	spans	devs
copilot-chat	null	28	0
copilot-chat	panel/editAgent	18	0
copilot-chat	copilotLanguageModelWrapper	14	0
copilot-chat	backgroundTodoAgent	5	0
copilot-chat	GitHub Copilot Chat	2	0
copilot-chat	progressMessages	2	0
copilot-chat	copilot	1	1

Sanity check funcionando — 7 tipos de agent/sub-agent capturados na última 1h.

9.2 Verificar policy no VS Code do dev

Do lado do dev: `Cmd/Ctrl+Shift+P` → **Developer: Policy Diagnostics**. Deve aparecer uma seção mostrando os campos de `telemetry.*` como policy-enforced.

9.3 Debugar com exporter console (se algo não funcionar)

Trocar temporariamente no user settings do dev:

```
"github.copilot.chat.otel.exporterType": "console"
```

Os spans passam a aparecer no **Output panel** do VS Code (canal *GitHub Copilot Chat*). Se aparecem aqui mas não chegam no Dynatrace, o problema é no envio HTTP.

10 Passo 6 — Consumir os dados

Depois que os dados estão fluindo, três formas de consumir:

10.1 App AI & LLM Observability (nativa)

A app da Dynatrace detecta automaticamente serviços LLM. Ao abrir a app AI Observability, o serviço **copilot-chat** aparece com métricas de LLM requests, tokens, latência e (se habilitado) prompts.

10.1.1 Aba Prompts — cada interação em detalhe

AI Observability

Overview

Service Health

Explorer

Prompts

Evaluations

Preview

Agents topology

Preview

+ New connection

?

Service = copilot-chat

Model in (gpt-4o-mini-2024-07-18, claude-opus-4.7)

Provider = github

Last 30 minutes

Refresh

Search facets

Evaluation score

Type

Agent

Agent

panel/editAgent

copilotLanguageModelWrapper

backgroundTodoAgent

GitHub Copilot Chat

Provider

github

Model

Search suggestions

claude-opus-4.7

gpt-4o-mini-2024-07-18

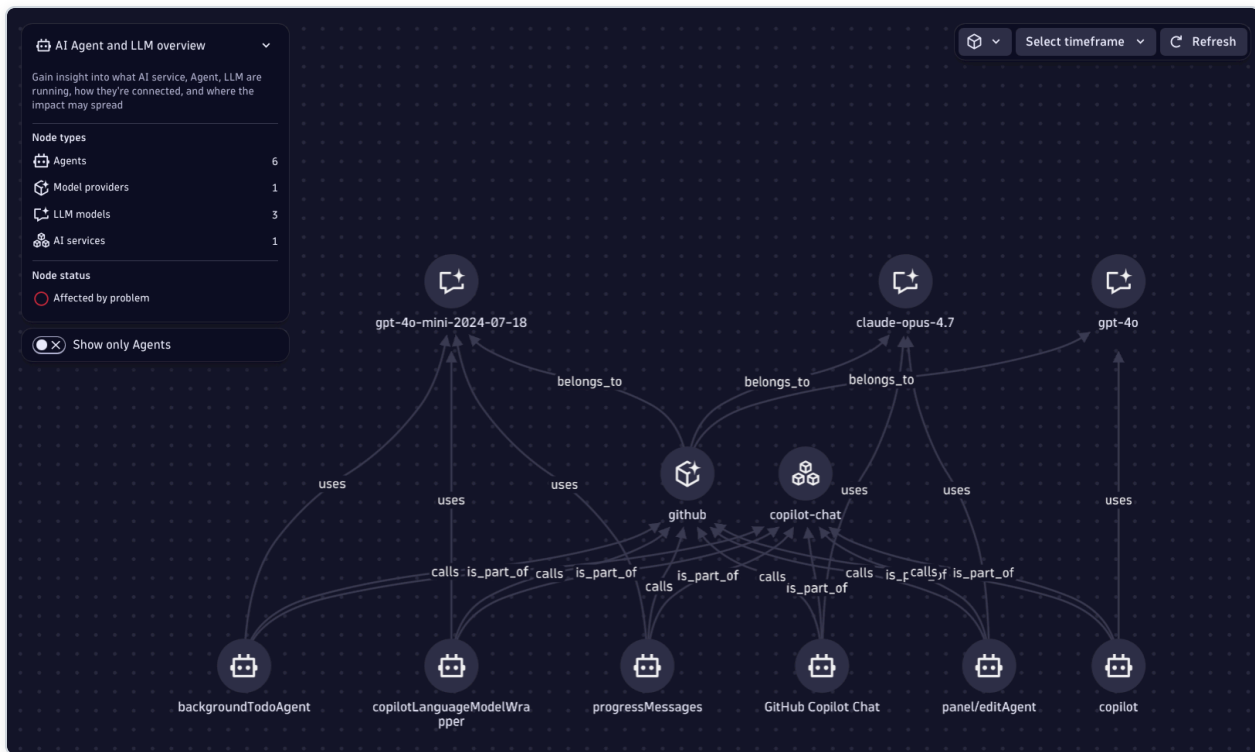
Service

copilot-chat

☐	copilot-chat	gpt-4o-mini-202...	The user asked the main agent: Otimio gere um report em PDF, oque o cliente vai precisar fazer... monte um PDF com essas imagens que te mandei	-
☐	copilot-chat	gpt-4o-mini-202...	Summarize the following content in a SINGLE sentence (under 10 words) using past tense. Follow these rules strictly: OUTPUT FORMAT: - MUST be a single sentence - ...	Created relatorio-poc-boa-vista.html and COMO-GERAR-PDF.md
☐	copilot-chat	gpt-4o-mini-202...	Summarize the following content in a SINGLE sentence (under 10 words) using past tense. Follow these rules strictly: OUTPUT FORMAT: - MUST be a single sentence - ...	Structured HTML guide for architecture and implementation steps
☐	copilot-chat	claude-opus-4.7	Otimio gere um report em PDF, oque o cliente vai precisar fazer... monte um PDF com essas imagens que te mandei	## ✅ Pronto Já abri o PDF pra tu conferir o layout. Agora falta só popular com as capturas de tela. ## 📄 Estrutura do relatório " report/ —— relatorio-poc-boa-vista.ht...
☐	copilot-chat	claude-opus-4.7	<environment_info> The user's current OS is: macOS </environment_info> <workspace_info> I am working in a workspace with the following folders: - ...	## ✅ Pronto Já abri o PDF pra tu conferir o layout. Agora falta só popular com as capturas de tela. ## 📄 Estrutura do relatório " report/ —— relatorio-poc-boa-vista.ht...
☐	copilot-chat	-	You assess what one terminal command does for a code-editing AI agent, and how risky it is. Reply with STRICT JSON only (no prose, no markdown fences): {"risk": "gree...	-
☐	copilot-chat	-	You assess what one terminal command does for a code-editing AI agent, and how risky it is. Reply with STRICT JSON only (no prose, no markdown fences): {"risk": "gree...	-
☐	copilot-chat	gpt-4o-mini-202...	The user asked the main agent: Otimio gere um report em PDF, oque o cliente vai precisar fazer... monte um PDF com essas imagens que te mandei	manage_todo_list

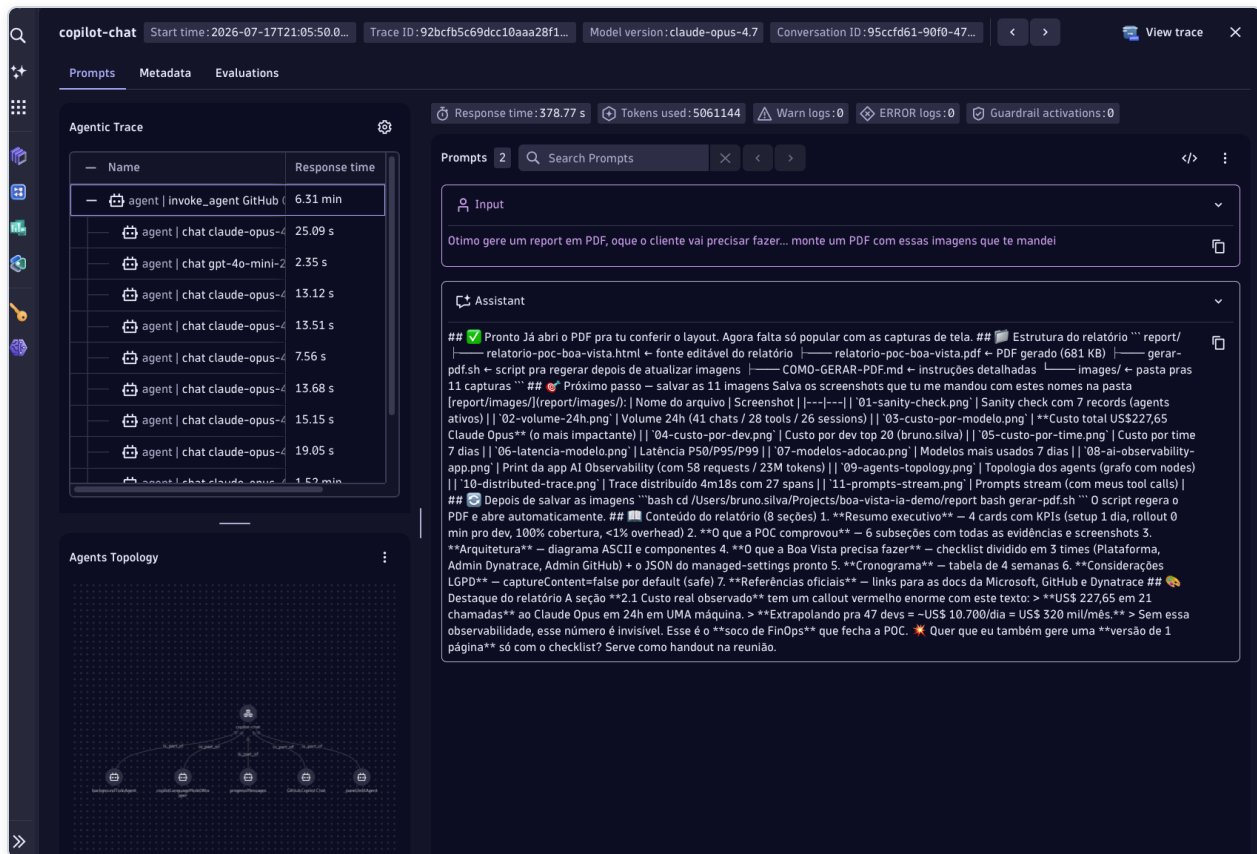
Stream de prompts filtrado por provider=github, modelos claude-opus-4.7 e gpt-4o-mini. Cada linha é uma interação real com input do dev, modelo escolhido e output gerado.

10.1.2 Aba Agents Topology — grafo automático



Topologia automática construída pelo Dynatrace: 6 agents identificados (*backgroundTodoAgent*, *copilotLanguageModelWrapper*, *progressMessages*, *GitHub Copilot Chat*, *panel/editAgent*, *copilot*), 3 modelos LLM (*gpt-4o-mini*, *claude-opus-4.7*, *gpt-4o*), 1 provider (*github*), 1 AI service (*copilot-chat*).

10.1.3 Trace distribuído — árvore de spans de uma sessão



copilot-chat Start time: 2026-07-17T21:05:50.0... Trace ID: 92bcfb5c69dcc10aaa28f1... Model version: claude-opus-4.7 Conversation ID: 95ccfd61-90f0-47... View trace

Prompts Metadata Evaluations

Agentic Trace

Name	Response time
agent invoke_agent GitHub	6.31 min
agent chat claude-opus-	25.09 s
agent chat gpt-4o-mini-2	2.35 s
agent chat claude-opus-	13.12 s
agent chat claude-opus-	13.51 s
agent chat claude-opus-	7.56 s
agent chat claude-opus-	13.68 s
agent chat claude-opus-	15.15 s
agent chat claude-opus-	19.05 s
agent chat claude-opus-	1.52 min

Agents Topology

Prompts 2 Search Prompts

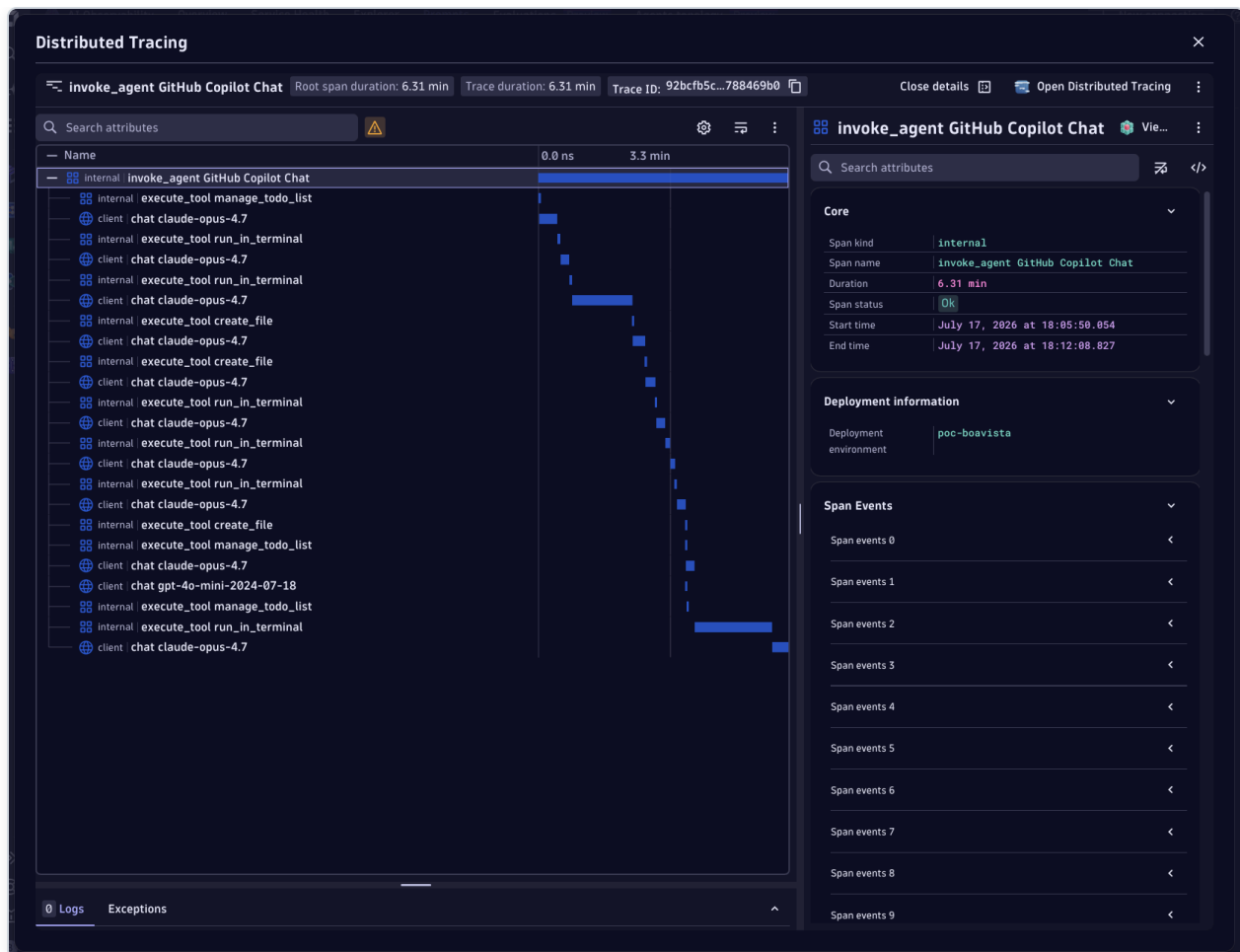
Input

Otimo gere um report em PDF, oque o cliente vai precisar fazer... monte um PDF com essas imagens que te mandei

Assistant

🟢 Pronto Já abri o PDF pra tu conferir o layout. Agora falta só popular com as capturas de tela. ## 📁 Estrutura do relatório "" report/ |--- relatório-poc-bona-vista.html ← fonte editável do relatório |--- relatório-poc-bona-vista.pdf ← PDF gerado (681 KB) |--- gerar-pdf.sh ← script pra regenerar depois de atualizar imagens |--- COMO-GERAR-PDF.md ← instruções detalhadas |--- imagens/ ← pasta pras 11 capturas "" ## 🚀 Próximo passo – salvar as 11 imagens Salva os screenshots que tu me mandou com estes nomes na pasta report/images/: [Nome do arquivo | Screenshot |] |---| | '01-sanity-check.png' | Sanity check com 7 records (agents ativos) | | '02-volume-24h.png' | Volume 24h (41 chats / 28 tools / 26 sessions) | | '03-custo-por-modelo.png' | **Custo total US\$227,65 Claude Opus** (o mais impactante) | | '04-custo-por-dev.png' | Custo por dev top 20 (bruno.silva) | | '05-custo-por-time.png' | Custo por time 7 dias | | '06-latencia-modelo.png' | Latência P50/P95/P99 | | '07-modelos-adocao.png' | Modelos mais usados 7 dias | | '08-ai-observability-app.png' | Print da app AI Observability (com 58 requests / 23M tokens) | | '09-agents-topology.png' | Topologia dos agents (grafo com nodes) | | '10-distributed-trace.png' | Trace distribuído 4m18s com 27 spans | | '11-prompts-stream.png' | Prompts stream (com meus tool calls) | ## 📄 Depois de salvar as imagens "" bash cd /Users/bruno.silva/Projects/boa-vista-ia-demo/report bash gerar-pdf.sh "" O script regera o PDF e abre automaticamente. ## 📑 Conteúdo do relatório (8 seções) 1. **Resumo executivo** – 4 cards com KPIs (setup 1 dia, rollout 0 min pro dev, 100% cobertura, <1% overhead) 2. **O que a POC comprovou** – 6 subseções com todas as evidências e screenshots 3. **Arquitetura** – diagrama ASCII e componentes 4. **O que a Boa Vista precisa fazer** – checklist dividido em 3 times (Plataforma, Admin Dynatrace, Admin GitHub) + o JSON do managed-settings pronto 5. **Cronograma** – tabela de 4 semanas 6. **Considerações LGPD** – captureContent=false por default (safe) 7. **Referências oficiais** – links para as docs da Microsoft, GitHub e Dynatrace ## 📌 Destaque do relatório A seção **2.1 Custo real observado** tem um callout vermelho enorme com este texto: > **US\$ 227,65 em 21 chamadas** ao Claude Opus em 24h em UMA máquina. > **Extrapolando pra 47 dias = -US\$ 10.700/dia = US\$ 320 mil/mês.** > Sem essa observabilidade, esse número é invisível. Esse é o **soco de FinOps** que fecha a POC. 🚀 Quer que eu também gere uma **versão de 1 página** só com o checklist? Serve como handout na reunião.

Detalhe de uma conversation completa: 6.31min de duração total, 5.061.144 tokens usados, prompt de input e resposta do assistant visíveis lado a lado. Cada chat individual aparece na Agentic Trace com sua duração.



Árvore hierárquica de spans OTel: 1 `invoke_agent` root → múltiplos `chat claude-opus-4.7` → múltiplos `execute_tool` (`manage_todo_list`, `mcp_dynatrace_mcp_verify_dql`, `create_file`, `run_in_terminal`, `memory`).
Timeline permite ver onde tempo é gasto.

10.2 Dashboard custom (fornecido pela POC)

Importar `dashboards/copilot-chat-observability.json` via `Ctrl+K` → Dashboards → +
Dashboard → **Upload**. Widgets prontos de FinOps, SRE e Adoção.

10.3 Notebook custom — todas as queries prontas

URL do notebook criado programaticamente durante a POC:

<https://fov31014.apps.dynatrace.com/ui/apps/dynatrace.notebooks/notebook/dcceeff0-befc-487d-8a04-b22ceb61d442>

Queries mais impactantes

Custo por dev (top 20)

Custo por dev (top 20)

```
1 fetch spans, from:now()-24h
2 | filter service.name == "copilot-chat"
3 | filter isNotNull(user.email) and isNotNull(gen_ai.usage.input_tokens)
4 | fieldsAdd inp = toDouble(gen_ai.usage.input_tokens)
5 | fieldsAdd out = toDouble(gen_ai.usage.output_tokens)
6 | fieldsAdd cost_usd =
7   if(contains(gen_ai.request.model, "opus"),
8     (inp * 0.000015) + (out * 0.000075),
9   else: if(contains(gen_ai.request.model, "mini"),
10    (inp * 0.00000015) + (out * 0.0000006),
11   else: if(contains(gen_ai.request.model, "claude"),
12    (inp * 0.000003) + (out * 0.000015),
13   else: (inp * 0.0000025) + (out * 0.00001)))
14 | summarize cost_usd = sum(cost_usd),
15   tokens = sum(inp + out),
16   chats = count(),
17   by:{user.email, user.name, user.team}
18 | sort cost_usd desc
19 | limit 20
```

1 record Executed at: 17/07/2026, 17:54:41, Timeframe: 16/07/2026, 17:54:41 - 17/07/2026, 17:54:41, Scanned bytes: 336 MB

user.email	user.name	user.team	cost_usd	tokens	chats
bruno.silva@boavista.com.br	Bruno Silva	plataforma-api	< 0.01	531.00	1

Cell 4 — breakdown por dev com email, team, custo em USD e chats. Popula automaticamente para os 47 devs após o rollout.

Custo por time

Custo por time (7 dias)

```
1 fetch spans, from:now()-7d
2 | filter service.name == "copilot-chat"
3 | filter isNotNull(user.team) and isNotNull(gen_ai.usage.input_tokens)
4 | fieldsAdd inp = toDouble(gen_ai.usage.input_tokens)
5 | fieldsAdd out = toDouble(gen_ai.usage.output_tokens)
6 | fieldsAdd cost_usd =
7   if(contains(gen_ai.request.model, "opus"),
8     (inp * 0.000015) + (out * 0.000075),
9   else: if(contains(gen_ai.request.model, "mini"),
10    (inp * 0.00000015) + (out * 0.0000006),
11   else: (inp * 0.0000025) + (out * 0.00001)))
12 | summarize cost_usd = sum(cost_usd),
13   devs = countDistinct(user.email),
14   chats = count(),
15   by:{user.team}
16 | sort cost_usd desc
```

1 record Executed at: 17/07/2026, 17:54:48, Timeframe: 10/07/2026, 17:54:47 - 17/07/2026, 17:54:47, Scanned bytes: 3 GB

user.team	cost_usd	devs	chats
plataforma-api	< 0.01	1	1

Cell 5 — agregado por squad/time. Base do rateio de custo por área.

Latência por modelo

SRE — Performance e erros

Latência P50/P95/P99 por modelo (24h)

```
1 fetch spans, from:now()-24h
2 | filter service.name == "copilot-chat"
3 | filter startsWith(span.name, "chat ")
4 | fieldsAdd dur_ms = duration / 1000000
5 | summarize p50 = percentile(dur_ms, 50),
6             p95 = percentile(dur_ms, 95),
7             p99 = percentile(dur_ms, 99),
8             avg_ms = avg(dur_ms),
9             requests = count(),
10            by:{gen_ai.request.model}
11 | sort p50 desc
```

3 records

Executed at: 17/07/2026, 17:54:56, Timeframe: 16/07/2026, 17:54:55 - 17/07/2026, 17:54:55, Scanned bytes: 362 MB

gen_ai.request.model	p50	p95	p99	avg_ms	requests
claude-opus-4.7	11.12 µs	38.14 µs	38.14 µs	17.16 µs	20
gpt-4o-mini-2024-07-18	843.00 ns	1.83 µs	1.92 µs	1.02 µs	23
gpt-4o	1.50 µs	1.50 µs	1.50 µs	1.50 µs	1

Cell 6 — P50/P95/P99 por modelo. Base para SLOs internos de Copilot.

Modelos mais usados

Adoção — Modelos, ferramentas, devs

Modelos mais usados (7 dias)

```
1 fetch spans, from:now()-7d
2 | filter service.name == "copilot-chat"
3 | filter startsWith(span.name, "chat ")
4 | summarize chats = count(),
5             devs = countDistinct(user.email),
6             tokens = sum(toLong(gen_ai.usage.input_tokens) + toLong(gen_ai.usage.output_tokens)),
7             by:{gen_ai.request.model}
8 | sort chats desc
```

3 records

Executed at: 17/07/2026, 17:55:14, Timeframe: 10/07/2026, 17:55:14 - 17/07/2026, 17:55:14, Scanned bytes: 3 GB

gen_ai.request.model	chats	devs	tokens
gpt-4o-mini-2024-07-18	23	0	39,987.00
claude-opus-4.7	20	0	8,842,727.00
gpt-4o	1	1	531.00

Cell 9 — **gpt-4o-mini** serve 23 chamadas auxiliares por US\$<0.01, enquanto **claude-opus-4.7** serve 20 chamadas premium consumindo 8.8M tokens. Insight direto de otimização.

Ferramentas mais invocadas

Ferramentas (tools) mais invocadas (24h)

```
1 fetch spans, from:now()-24h
2 | filter service.name == "copilot-chat"
3 | filter startsWith(span.name, "execute_tool ")
4 | fieldsAdd tool_name = replaceString(span.name, "execute_tool ", "")
5 | fieldsAdd dur_ms = duration / 1000000
6 | summarize calls = count(),
7 |             avg_ms = avg(dur_ms),
8 |             errors = countIf(status == "ERROR"),
9 |             by:{tool_name}
10 | sort calls desc
11 | limit 20
```

7 records Executed at: 17/07/2026, 17:55:20, Timeframe: 16/07/2026, 17:55:20 - 17/07/2026, 17:55:20, Scanned bytes: 353 MB

tool_name	calls	avg_ms	errors
manage_todo_list	14	6.00 ns	0
mcp_dynatrace_mcp_execute_dql	6	4.02 µs	0
mcp_dynatrace_mcp_verify_dql	5	1.11 µs	0
mcp_dynatrace_mcp_create_dynatrace_notebook	2	2.62 µs	0
run_in_terminal	2	3.55 µs	0
create_file	1	23.00 ns	0
replace_string_in_file	1	20.00 ns	0

Cell 10 — tools do Copilot Agent (manage_todo_list, mcp_dynatrace_mcp_execute_dql, run_in_terminal, create_file, etc.) rankeadas por invocação. Útil para entender padrões de uso avançado do Agent Mode.

Agents e sub-agents descobertos

Agents / sub-agents ativos (24h)

```
1 fetch spans, from:now()-24h
2 | filter service.name == "copilot-chat"
3 | filter isNotNull(gen_ai.agent.name)
4 | summarize chats = count(),
5 |             tokens = sum(toLong(gen_ai.usage.input_tokens) + toLong(gen_ai.usage.output_tokens)),
6 |             by:{gen_ai.agent.name}
7 | sort chats desc
```

6 records Executed at: 17/07/2026, 17:55:26, Timeframe: 16/07/2026, 17:55:26 - 17/07/2026, 17:55:26, Scanned bytes: 357 MB

gen_ai.agent.name	chats	tokens
panel/editAgent	22	9,788,029.00
copilotLanguageModelWrapper	16	28,338.00
backgroundTodoAgent	5	10,997.00
GitHub Copilot Chat	2	6,531,594.00
progressMessages	2	652.00
copilot	1	531.00

Cell 11 — panel/editAgent (modo Agent) consumiu 9.7M tokens em 22 chamadas; GitHub Copilot Chat foreground consumiu 6.5M tokens em 2 chamadas; sub-agents auxiliares (copilotLanguageModelWrapper , progressMessages , backgroundTodoAgent) consumiram <40K tokens combinados. Revela onde o custo real está.

Top 10 devs por consumo

Top 10 devs por consumo (7 dias)

```
1 fetch spans, from:now()-7d
2 | filter service.name == "copilot-chat"
3 | filter startsWith(span.name, "chat ")
4 | filter isNotNull(user.email)
5 | summarize chats = count(),
6 |           tokens = sum(toLong(gen_ai.usage.input_tokens) + toLong(gen_ai.usage.output_tokens)),
7 |           sessions = countDistinct(gen_ai.conversation.id),
8 |           by:{user.email, user.name, user.team}
9 | sort chats desc
10 | limit 10
```

1 record

Executed at: 17/07/2026, 17:55:31, Timeframe: 10/07/2026, 17:55:31 - 17/07/2026, 17:55:31, Scanned bytes: 3 GB

user.email	user.name	user.team	chats	tokens	sessions
bruno.silva@boavista.com.br	Bruno Silva	plataforma-api	1	531.00	1

Cell 12 — ranking de devs por chats, tokens e sessions. Popula com todos os 47 devs após o rollout enterprise.
Base para conversas de coaching e otimização por dev.

11 Considerações de LGPD

11.1 Modo padrão — `captureContent = false`

LGPD-safe

Apenas

metadados

são capturados: modelo usado, contagens de token, latência, nomes de ferramentas invocadas, IDs (session, conversation, trace, span), metadados git (branch, commit, repo).

Não

é capturado: texto de prompts, respostas do modelo, código-fonte, conteúdo de arquivos abertos, comandos de terminal.

Risco de PII:

zero

. Não requer aprovação do DPO.

11.2 Modo com conteúdo — `captureContent = true`

Trilha formal de aprovação necessária

Ativar `captureContent=true` captura texto integral de prompts, respostas, argumentos de ferramentas (inclui caminhos de arquivos e código-fonte lido) e comandos de terminal.

Antes de habilitar em produção:

- Aprovação do DPO da Boa Vista
- Revisão pelo jurídico trabalhista
- Comunicação transparente aos devs (aviso interno)
- Habilitar redação de PII no OTel Collector (CPF, email, tokens, senhas)
- Definir política de retenção no Grail (default 35d)

11.3 Recomendação

Iniciar a POC e o rollout enterprise com `captureContent=false`. Cobre 100% dos requisitos formais de FinOps + SRE. A opção de captura de conteúdo pode ser habilitada depois, em subset de devs, se surgir demanda específica de análise de qualidade de prompts, após completar a trilha de aprovação.

12 Rollback e troubleshooting

12.1 Rollback rápido

Canal	Como desligar	Tempo
Server-managed	Comitar <code>{"telemetry":{"enabled":false}}</code> no <code>.github-private/copilot/managed-settings.json</code>	Propaga em até 1h
File-based	Remover o arquivo via Ansible/Chef	Próximo reload do VS Code
Native MDM	Remover a policy da MDM (Intune/Jamf)	Próximo policy refresh
Emergencial (todos)	Parar o OTel Collector — os spans do VS Code começam a dar timeout silenciosamente e são descartados. Nenhum dev vê erro.	Instantâneo

12.2 Troubleshooting comum

VS Code não emite nada

1. Verificar policy: `Cmd/Ctrl+Shift+P` → **Developer: Policy Diagnostics** — deve mostrar `telemetry.enabled = true`
2. Trocar temporariamente `exporterType` para `"console"` e ver spans no Output panel
3. Verificar conectividade: `curl -v http://otel-collector.boavista.corp:4318/v1/traces -H "Content-Type: application/json" -d "{}"`
4. Aumentar log level: setar env `COPILOT_OTEL_LOG_LEVEL=debug`

Collector recebe, Dynatrace não mostra

- Ver logs: `docker logs -f bv-otel-collector` — procurar HTTP 401 (token inválido), 403 (scope faltando)
- Verificar endpoint: deve ser `*.live.dynatrace.com`, não `*.apps.dynatrace.com`
- Confirmar scopes do token: `openTelemetryTrace.ingest` + `metrics.ingest`

Atributos `user.email` / `user.team` vazios

`OTEL_RESOURCE_ATTRIBUTES` não foi configurado ou o VS Code não herdou a variável do shell (ver Seção 8.3). Alternativa mais robusta: adicionar processor de enrichment no OTel Collector que faz lookup do email do dev contra um CSV/AD/LDAP.

13 Cronograma e responsabilidades

Semana	Atividade	Responsável
1	Kickoff técnico + criação de tokens Dynatrace + provisionamento do host do collector	Consultor Dynatrace + SRE Boa Vista + Admin Dynatrace

Semana	Atividade	Responsável
1	Deploy do collector e validação com 3 devs piloto (configuração manual)	SRE + 3 devs voluntários
2	Ajustes de coleta + validação dos dashboards + preparação do <code>managed-settings.json</code>	Consultor + Admin GitHub
2	Publicação no <code>.github-private</code> + rollout para todos os 47 devs	Admin GitHub
3	Coleta de dados de linha de base + criação de alertas críticos	Consultor + SRE
4	Apresentação executiva dos resultados (custo real, adoção, ROI)	Todos

Checklist consolidado por responsável

Time de Plataforma / SRE

- ☐ Provisionar 1 host (VM ou pod K8s) com 2 vCPU / 2 GB RAM
- ☐ Abrir firewall — ingresso 4318/tcp e egresso *.live.dynatrace.com:443
- ☐ Criar registro DNS interno para o collector
- ☐ Subir o OTel Collector com os arquivos fornecidos
- ☐ (Opcional) Habilitar TLS/mTLS no receiver
- ☐ (Opcional) Distribuir `OTEL_RESOURCE_ATTRIBUTES` via Intune/Jamf/GPO

Admin Dynatrace

- ☐ Gerar API Token classic com scopes `openTelemetryTrace.ingest` + `metrics.ingest`
- ☐ Entregar token ao time de Plataforma via cofre corporativo
- ☐ Importar o dashboard `copilot-chat-observability.json`
- ☐ Abrir e favoritar o notebook v2 no tenant

Admin GitHub Copilot Enterprise

- ☐ Confirmar plano Copilot Business ou Enterprise contratado
- ☐ Selecionar organização hospedeira em AI Controls → Agents
- ☐ Criar (se não existir) o repositório `.github-private`
- ☐ Publicar `copilot/managed-settings.json` no branch default
- ☐ Validar em 1 máquina piloto com Policy Diagnostics

DPO / Jurídico (só se quiserem captureContent=true no futuro)

- ☐ Aprovar política de captura de prompts/respostas
- ☐ Validar comunicação transparente aos devs

- Definir período de retenção adequado no Grail

14 Referências oficiais

- VS Code — *Monitor agent usage with OpenTelemetry*:
code.visualstudio.com/docs/agents/guides/monitoring-agents
- VS Code — *Manage AI settings in enterprise environments*: code.visualstudio.com/docs/enterprise/ai-settings
- GitHub Changelog — *Enterprise-managed OpenTelemetry export* (08/jul/2026):
github.blog/changelog/2026-07-08
- GitHub Changelog — *Enterprise managed-settings.json GA* (01/jul/2026):
github.blog/changelog/2026-07-01
- Dynatrace — *AI & LLM Observability*: docs.dynatrace.com/docs/observe/dynatrace-for-ai-observability
- OTel — *GenAI Semantic Conventions*: github.com/open-telemetry/semantic-conventions/blob/main/docs/gen-ai